

CK PIN Code Master — data pipeline for the CK App & System

Own the data, rent nothing. This repo turns free Government of India postal data into a normalized, versioned, audit-traceable PIN code master with geometry, plus a monthly diff so the dataset does not quietly rot.

What is in the box

```
ck_pincode/
├─ schema.sql           normalized SQLite schema (Postgres/PostGIS notes inline)
├─ fetch_datagov.py     resumable, rate-limit-aware puller for data.gov.in resources
├─ build_geo.py         boundary GeoJSON -> centroids, bboxes, simplified polygons
├─ build_db.py          full rebuild of the master DB from raw inputs
├─ refresh.py           monthly snapshot + field-level diff + change_log + report
├─ export.py            app-ready CSV/JSON exports + QA summary
├─ raw/                 immutable source payloads (never edit by hand)
├─ out/
│   ├── ck_pincode.db   the master (120 MB, includes polygons)
│   ├── QA_SUMMARY.md   coverage and integrity report
│   ├── pincode_geo.csv
│   ├── pincode_boundaries.geojson      219 MB, full fidelity
│   ├── pincode_boundaries.simplified.geojson  54 MB, for map rendering
│   └── exports/        flat files for seeding the app
└─ reports/             dated change reports from each refresh
```

Current state of the master

Metric	Value
Active PIN codes (have a live post office)	19,100
Deliverable PIN codes	19,093
Boundary-only / retired PIN codes	836
Distinct PIN codes tracked (active + retired)	19,936
Post offices	154,781
PIN codes with boundary polygon + centroid	19,928
PIN codes spanning more than one district	1,246
PIN codes spanning more than one state	34
States + UTs / districts	36 / 632

The 19,100 active figure reconciles with the published national count of 19,101 PIN codes, which is the

cross-check that matters — the pipeline is not inventing or dropping PINs.

Full detail: `out/QA_SUMMARY.md`.

Sources and how much to trust them

Source	What it gives	Authoritative?	Link
All India Pincode Directory till last month (Dept. of Posts, resource <code>5c2f62fe-5afa-4119-a499-fec9d604d5bd</code> , 165,627 rows, lat/long included)	the master office list — use this for anything regulated	Yes	data.gov.in
LGD Villages with PIN Codes (<code>f17a1608-5f10-4610-bb50-a63c80d83974</code> , 677,323 rows, refreshed Sept 2026)	village → PIN, replaces the paid locality databases	Yes	data.gov.in
LGD Local Bodies with PIN Codes (<code>71818d1a-c114-46cb-aa9b-56ed70d4bc4a</code> , 7,411 rows)	urban local body → PIN	Yes	data.gov.in
All India Pincode Boundary GeoJSON	official polygons per delivery office	Yes	data.gov.in
PIN boundary polygon compilation (19,928 polygons, used here)	geometry loaded into this build	No — verify before regulatory use	er-data-storage/postal-code-data
Bulk directory CSV snapshot (154,797 rows, used to bootstrap this build)	office list currently loaded	No — older vintage	saravanakumargn mirror
DIGIPIN	India Post's new 10-character geo-code addressing grid	Yes	India Post

⚠ One action required from you (2 minutes)

The office layer in this build came from the **bulk snapshot**, not the live government feed. Reason: `data.gov.in` throttles the shared public sample key to 10 rows per request and it is usually rate-limited to zero, so a 165,627-row pull is impossible with it.

1. Register free at data.gov.in
2. My Account → API key
3. Then:

```
export DATA_GOV_KEY=<your key>
python fetch_datagov.py 5c2f62fe-5afa-4119-a499-fec9d604d5bd raw/postoffices.jsonl
python build_db.py && python export.py
```

With your own key the page size jumps to 1,000+ rows, so the full pull takes **under two minutes** and `build_db.py` automatically prefers

raw/postoffices.jsonl over the snapshot. That single step converts this from a working prototype into an audit-grade master with official lat/long per office.

The same command loads the 677k-row village layer:

```
python fetch_datagov.py f17a1608-5f10-4610-bb50-a63c80d83974 raw/lgd_villages.jsonl
python build_db.py      # picks up the villages file automatically
```

Schema, and why it is shaped this way

post_office is the grain of truth. India Post publishes *offices*, not PIN codes. Every "how many PIN codes" number in the country is a derived rollup, which is exactly why published counts disagree (19,097 / 19,101 / 19,300).

pincode is derived. Rebuilt from post_office on every refresh. Carries n_offices, n_districts, n_states, deliverability, centroid, bbox, area.

locality_pincode is many-to-many. A village maps to a PIN and a PIN covers many villages. Storing locality on the PIN row is the single most common modelling mistake here and it silently breaks address validation.

Nothing is hard-deleted. Rows close with valid_to instead. A CK customer address captured in 2024 must still resolve in 2027 even if the office shut — non-negotiable for KYC and audit trails.

Every row carries source_id + snapshot_id. Any single value can be traced back to a dated government file. This is the part that matters if CK is ever inspected.

name_alias resolves raw spellings to canonical names. India Post spells the same district several ways and state reorganizations leave stale names behind (this build still shows Telangana offices under "Andhra Pradesh" circle, which is exactly the class of rot the alias table absorbs).

App-facing views

View	Use
v_pincode_lookup	the single call your checkout / onboarding form makes
v_pincode_offices	"select your nearest post office" UX
v_coverage_gaps	QA: 844 PINs with geometry but no live office, or vice versa

Monthly refresh

```
export DATA_GOV_KEY=<your key>
python refresh.py --dry-run      # see the delta before touching anything
python refresh.py                # apply + write reports/change_report_<date>.md
```

What it does: pulls a fresh snapshot, diffs it field by field against current rows, closes disappeared offices, inserts new ones, records every change in `change_log`, recomputes the PIN rollup, and writes a dated markdown report.

Safety guard: it refuses to apply a snapshot that would remove more than 2% of offices and exits 4. Truncated API pulls look exactly like a mass closure event, and silently accepting one would wipe live serviceability. Override with `--force` only after reading the blocked report.

Verified on a synthetic snapshot: 5 additions, 30 closures, 118 field changes → 321 `change_log` rows, and PIN-level status transitions recorded correctly.

Suggested cadence: monthly, a few days after the Department of Posts refresh (the dataset is titled "till last month" for a reason). Daily adds nothing.

Exports for the app

File	Rows	Use
<code>pincode_master.csv</code>	19,936	primary seed table
<code>post_offices.csv</code>	154,781	office picker, branch mapping
<code>pincode_centroids.csv</code>	19,928	distance / radius serviceability
<code>pincode_lookup.min.json</code>	19,100	1.0 MB compact LUT for edge cache or offline mode
<code>pincode_district_map.csv</code>	20,402	the many-to-many truth, use this over <code>primary_district</code>
<code>pincode_boundaries.simplified.geojson</code>	19,928	choropleths, coverage maps
<code>qa_multi_district_pincodes.csv</code>	1,246	the traps — review before trusting PIN → district
<code>states.csv,districts.csv</code>	36 / 632	reference dropdowns

Known caveats — read before shipping

1. **Office layer vintage.** Bootstrapped from the bulk snapshot; circle names are pre-reorganization in places. Fixed by the 2-minute API key step above.
2. **1,246 PIN codes span multiple districts and 34 span multiple states.** Never auto-fill district from PIN and treat it as verified. Use it as a default the user can override, and store what the user confirmed. This is the number one cause of failed address matches in Indian apps.

3. **PIN 853204 maps to 5 districts across 34 offices.** Test your address logic against `qa_multi_district_pincodes.csv`, not against Mumbai and Pune.
4. **Boundary polygons are a community compilation.** Good enough for maps, distance and serviceability. Swap in the official [data.gov.in boundary GeoJSON](https://data.gov.in/boundary-geojson) before using geometry in anything regulatory.
5. **3,942 offices have an unrecognized office type** in the snapshot. The API pull resolves nearly all of these.
6. **Zone 9 (Army Postal Service) PINs are flagged but absent** from the civil directory. If CK ever serves defence personnel, that is a separate feed.
7. **Licensing.** Government data is under the Government Open Data Licence – India (GODL): free commercial use, attribution required. Attribute "Department of Posts, Government of India, via data.gov.in" in the CK App about/legal screen. The community mirrors are derived works — the API key step removes any dependency on them.

Cost

Zero recurring, versus ₹4,999+/month for a commercial PIN code API. The only real cost is the engineer-week to wire `v_pincode_lookup` into the CK App and put `refresh.py` on a monthly schedule.